

SingularityFlow

Cross-Platform Build, Packaging & Installation

One universal CLI and Copilot plugin package. One universal VS Code extension. Verified installation paths for Windows and macOS.

Guide	Applies to	Primary audience
Build and installation	SingularityFlow 0.9.x	Release engineers, administrators and pilot users

Governing rule: End users install packaged artifacts. Bash is permitted for source contributors, but it is not part of the normal Windows or macOS product journey.

Prepared from current main and the repository's release scripts

1. Purpose and release outcome

This guide defines the supported procedure for compiling SingularityFlow, producing its two universal artifacts, installing them on Windows and macOS, and proving that the CLI, Copilot plugin and VS Code extension agree on the installed version.

Release outcome: A clean dist/ directory containing the universal npm tarball, universal VSIX, SHA256SUMS and RELEASE.json. The same artifacts are installed on Windows and macOS.

1.1 Artifact model

Artifact	Contains	Platform scope
singularity-flow- <version>.tgz	Node kernel, CLI, Copilot plugin, agents, skills, hooks, schemas and templates	Windows, macOS and Linux
singularity-flow-vscode- <version>.vsix	VS Code UI plus the matching staged CLI	Windows, macOS and Linux
SHA256SUMS	Integrity digest for every release artifact	Verification on every platform
RELEASE.json	Version, source commit and release metadata	Traceability and audit

Do not: Run vsce package directly. The repository release path stages the matching CLI inside the VSIX; a direct VSCE package can install successfully but leave the extension unable to execute lifecycle commands.

2. Supported operating model

Normal users install the packaged artifacts. They do not clone the SingularityFlow source tree and do not run install.sh. Source installation remains available for maintainers and contributors.

Journey	Windows	macOS
Normal product installation	PowerShell or Windows Terminal	Terminal (zsh or bash)
VS Code experience	Same VSIX	Same VSIX
Copilot plugin	Installed from the same npm package	Installed from the same npm package
Source-maintainer installation	Git Bash wrapper	install.sh from Terminal
Minimal world-model helper	Git for Windows Bash currently required	System shell available

3. Prerequisites

Requirement	Release machine	Installation machine
Node.js	22.6 or newer recommended; package minimum is 20	20 or newer

Requirement	Release machine	Installation machine
npm	From the selected Node installation	From the selected Node installation
Git	Required, with configured user identity	Git for Windows or Git for macOS
VS Code	Required to build and verify the extension	VS Code 1.90 or newer
Copilot CLI	Required for plugin validation	Required unless installing CLI-only
Registry access	Public npm or approved Artifactory	Access to the published tarball

VERIFY PREREQUISITES

```
node --version
npm --version
git --version
code --version
copilot --version
```

Build recommendation: Use Node.js 22.6 or newer for release builds. The runtime supports Node.js 20, but the full VS Code source-test path uses Node's TypeScript stripping capability when available.

4. Prepare a clean release checkout

1. **Use the intended release branch** - Normally main, or an approved release branch.
2. **Update without rewriting history** - Use a fast-forward-only pull.
3. **Confirm a clean tree** - The release script refuses uncommitted changes.
4. **Confirm the version** - package.json, the plugin manifest and VS Code manifest must agree.

MACOS OR GIT BASH

```
git switch main
git pull --ff-only
git status --short
git rev-parse HEAD
node -p "require('./package.json').version"
```

WINDOWS POWERSHELL

```
git switch main
git pull --ff-only
git status --short
git rev-parse HEAD
node -p "require('./package.json').version"
```

5. Compile and validate

The preferred path is the repository's release script. It performs deterministic checks before packaging, which is safer than assembling the CLI, plugin and extension separately.

5.1 Configure an approved npm registry

Credentials belong in the user's .npmrc or the organization's approved credential mechanism. Never place credentials in a registry URL, command history or release record.

WINDOWS POWERSHELL

```
$env:NPM_CONFIG_REGISTRY = "https://artifacts.company.example/api/npm/npm-virtual/"  
npm config get registry
```

MACOS TERMINAL

```
export NPM_CONFIG_REGISTRY="https://artifacts.company.example/api/npm/npm-virtual/"  
npm config get registry
```

5.2 Install exact dependencies and run checks

BOTH PLATFORMS

```
npm ci  
npm run check  
npm run test:all  
npm run vscode:typecheck  
npm run vscode:build  
npm run pack:dry
```

A failure stops the release. Do not bypass tests to obtain an artifact; the artifact would no longer represent the governed source commit recorded in the release metadata.

6. Produce the release artifacts

6.1 Dry run

BOTH PLATFORMS

```
npm run release:dry
```

Use the dry run to confirm packaging, versions and output names before cutting the distributable release.

6.2 Final release

BOTH PLATFORMS

```
npm run release
```

The command refuses a dirty tree, runs the repository checks and tests, builds the CLI tarball, stages the matching CLI into the VSIX, and writes release integrity metadata.

EXPECTED OUTPUT

```
dist/  
singularity-flow-<version>.tgz  
singularity-flow-vscode-<version>.vsix  
SHA256SUMS  
RELEASE.json
```

6.3 Individual artifact commands

Use these commands only for local development or targeted diagnosis. Use `npm run release` for distributable builds.

TARGETED PACKAGING

```
npm pack
npm run vscode:package
```

7. Verify artifact integrity

Before publishing or installing, compare the generated checksums with SHA256SUMS and retain `RELEASE.json` beside the artifacts.

WINDOWS POWERSHELL

```
Get-FileHash .\dist\*.tgz -Algorithm SHA256
Get-FileHash .\dist\*.vsix -Algorithm SHA256
Get-Content .\dist\SHA256SUMS
Get-Content .\dist\RELEASE.json
```

MACOS TERMINAL

```
shasum -a 256 dist/*.tgz dist/*.vsix
cat dist/SHA256SUMS
cat dist/RELEASE.json
```

Promotion rule: Publish the tarball, VSIX, SHA256SUMS and `RELEASE.json` as one release unit. Never mix artifacts produced from different commits or versions.

8. Install on Windows

8.1 Preferred packaged installation

Open PowerShell or Windows Terminal. A POSIX shell is not required for packaged installation.

POWERSHELL

```
npm install --global .\dist\singularity-flow-<version>.tgz
singularity-flow plugin install
code --install-extension .\dist\singularity-flow-vscode-<version>.vsix --force
```

npm creates Windows command shims such as `singularity-flow.cmd` and `sflow.cmd`. The Copilot hook has both bash and PowerShell forms and invokes the same installed kernel command.

8.2 Windows source-maintainer installation

Use this path only when installing directly from a clean SingularityFlow source checkout. Open Git Bash in the checkout:

GIT BASH

```
bash ./install-windows-git-bash.sh

# CLI only
bash ./install-windows-git-bash.sh --cli-only
```

```
# Corporate registry
bash ./install-windows-git-bash.sh --registry "https://artifacts.company.example/api/npm/npm-virtual/"
```

Windows limitation: The lightweight world-model wrapper currently delegates to a POSIX shell script. Git for Windows supplies the required Bash. Core lifecycle state, approvals, evidence and the VS Code UI do not depend on that script.

9. Install on macOS

9.1 Preferred packaged installation

TERMINAL

```
npm install --global ./dist/singularity-flow-<version>.tgz
singularity-flow plugin install
code --install-extension ./dist/singularity-flow-vscode-<version>.vsix --force
```

9.2 macOS source-maintainer installation

TERMINAL

```
bash ./install.sh

# CLI only
bash ./install.sh --cli-only

# Corporate registry
bash ./install.sh --registry "https://artifacts.company.example/api/npm/npm-virtual/"
```

macOS security: These artifacts are an npm package and a VS Code extension, not a native .app bundle. They do not require a DMG, notarization, an Apple developer certificate or a custom updater.

10. Validate the installation

1. **Confirm command resolution** - The command must resolve to the newly installed version.
2. **Confirm Copilot plugin registration** - Copilot should list singularity-flow.
3. **Confirm VS Code extension installation** - The extension identifier should appear in the installed list.
4. **Run product diagnostics** - Resolve every failure before beginning governed work.

WINDOWS POWERSHELL

```
where.exe singularity-flow
singularity-flow --version
copilot plugin list
code --list-extensions | Select-String -Pattern "singularity"
singularity-flow doctor --offline
```

MACOS TERMINAL

```
command -v singularity-flow
singularity-flow --version
copilot plugin list
code --list-extensions | grep -i singularity
singularity-flow doctor --offline
```

10.1 Repository smoke test

Perform the smoke test in a disposable repository or an approved pilot repository. Configure a local Git identity if the test machine does not already have one.

BOTH PLATFORMS

```
mkdir sflow-smoke
cd sflow-smoke
git init
git config user.name "SFlow Smoke Test"
git config user.email "sflow-smoke@example.invalid"
singularity-flow init
singularity-flow doctor --offline
```

11. Upgrade and reinstall

11.1 Normal in-place upgrade

BOTH PLATFORMS

```
npm install --global <new-path>/singularity-flow-<new-version>.tgz
singularity-flow plugin install
code --install-extension <new-path>/singularity-flow-vscode-<new-version>.vsix --force
singularity-flow doctor --offline
```

The `--force` option replaces the installed VS Code extension. Re-running `plugin install` aligns the Copilot plugin with the newly installed kernel.

11.2 Product-only clean reinstall

For a damaged or inconsistent installation, use the guarded reinstall planner from an approved checkout. Always preview first; the exact confirmation is generated from the preview.

GUARDED REINSTALL

```
sf-reinstall --checkout <absolute-path-to-singularityflow> --dry-run
sf-reinstall --checkout <absolute-path-to-singularityflow> --confirm "REINSTALL SINGULARITY FLOW <fingerprint>"
```

Preservation boundary: The product-only reinstall preserves application repositories, worktrees, governed state, credentials, settings and personal skills. It replaces only managed SFlow installation surfaces.

12. Troubleshooting

Symptom	Likely cause	Corrective action
sflow is not recognized	Global npm bin directory is not on PATH	Restart the terminal; inspect npm prefix; add the npm global bin location to PATH.
Copilot plugin is not listed	Plugin registration did not run or Copilot has stale state	Run <code>singularity-flow plugin install</code> , then restart Copilot CLI.
VS Code installs but commands fail	VSIX was built directly without staging the CLI	Rebuild with <code>npm run vscode:package</code> or <code>npm run release</code> .

Symptom	Likely cause	Corrective action
Agent descriptions are missing on Windows	Old build with CRLF-sensitive parsing	Install a current 0.9.x build containing the Windows CRLF/BOM fix.
npx quality command fails on Windows	The configured command resolved to npx.cmd through a shell-free path	Use a tested platform-aware command definition; do not approve the release until Windows CI passes.
World-model minimal reports no shell	Git Bash is unavailable	Install Git for Windows or use the Node-based world-model commands that do not require the legacy shell wrapper.
Corporate registry returns authentication errors	Missing or expired .npmrc credentials	Repair credentials using the organization's approved npm authentication flow; never embed secrets in the URL.
doctor reports build mismatch	CLI, plugin and extension came from different builds	Reinstall both artifacts from the same release directory and rerun doctor.

13. Cross-platform release gate

A release is ready for broader distribution only when all checks below pass on both a clean Windows runner and a clean macOS runner.

- The release checkout is clean and the source commit is recorded.
- npm ci completes from the approved registry.
- npm run check and npm run test:all pass.
- VS Code type-check and build pass.
- npm run release produces the tarball, VSIX, SHA256SUMS and RELEASE.json.
- The tarball installs globally and creates a working sflow command.
- Copilot lists the singularity-flow plugin after registration.
- The VSIX installs and opens the SingularityFlow navigation surface.
- singularity-flow doctor --offline reports no failures in the smoke repository.
- A CRLF checkout passes plugin/agent discovery on Windows.
- Windows quality commands exercise git, npm/npx and Copilot command resolution.
- The installed CLI, plugin and extension report the same version and source release.

Minimum certification: Do not call the product natively cross-platform until this gate runs automatically in CI on windows-latest and macos-latest.

14. Recommended CI matrix

Use one matrix to prove both the source build and the produced artifacts. The Windows job should run in PowerShell and explicitly test CRLF checkout behavior.

CI INTENT

```
strategy:
  matrix:
    os: [windows-latest, macos-latest, ubuntu-latest]
    node: [20, 22]

steps:
  - checkout
  - setup Node from matrix
  - npm ci
  - npm run check
  - npm run test:all
  - npm run vscode:typecheck
  - npm run release
  - install generated TGZ
  - register Copilot plugin
  - install generated VSIX
  - run cross-platform smoke test
```

15. Release handoff

Publish the complete release directory to the approved internal registry or artifact repository. The installation notice should provide the version, source commit, checksums, the two platform-neutral artifact names, supported Node/VS Code versions, and the exact installation and validation commands from this guide.

Handoff item	Required value
Version	The common version from package, plugin and VS Code manifests
Source	Full Git commit SHA and approved release branch
Artifacts	Universal TGZ and universal VSIX
Integrity	SHA256SUMS verified on the release machine
Evidence	Windows and macOS CI links or retained logs
Installation	PowerShell and macOS Terminal commands
Support boundary	Known world-model shell dependency and any open Windows command-resolution exception

Appendix A - Command card

Build

UNIVERSAL

```
npm ci
npm run release:dry
npm run release
```

Install

UNIVERSAL

```
npm install --global <artifact>/singularity-flow-<version>.tgz
singularity-flow plugin install
code --install-extension <artifact>/singularity-flow-vscode-<version>.vsix --force
```

Verify

UNIVERSAL

```
singularity-flow --version
copilot plugin list
code --list-extensions
singularity-flow doctor --offline
```

Appendix B - Maintained source references

These repository files are the normative implementation references for this guide:

- **DISTRIBUTION.md:** <https://github.com/ashokraj2011/singularityflow/blob/main/DISTRIBUTION.md>
- **package.json:** <https://github.com/ashokraj2011/singularityflow/blob/main/package.json>
- **Copilot plugin manifest:** <https://github.com/ashokraj2011/singularityflow/blob/main/plugin/plugin.json>
- **Cross-platform Copilot hooks:** <https://github.com/ashokraj2011/singularityflow/blob/main/plugin/hooks.json>
- **Windows Git Bash installer:** <https://github.com/ashokraj2011/singularityflow/blob/main/install-windows-git-bash.sh>
- **Windows CRLF/BOM correction:** <https://github.com/ashokraj2011/singularityflow/blob/main/docs/WINDOWS-GIT-BASH-INSTALL-FIX.md>